



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Construcción de máquinas de Estados usando memorias: Direccionamiento Entrada-Estado

NOMBRE DE LOS INTEGRANTES:
Omar Alfonso Zendejas Labias Grupo de Teoría: 04
Emiliano García Olvera Grupo de Teoría: 02

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06 EQUIPO: 9

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 22 de octubre de 2026

Objetivo

Construir una máquina de estados finita a partir de una carta ASM utilizando una memoria como elemento central del diseño, ahora bajo el método de direccionamiento entrada-estado. A diferencia del direccionamiento por trayectoria empleado en la práctica anterior, aquí la dirección de memoria se forma únicamente con el estado presente: la palabra almacenada indica cuál de las entradas debe sensarse y ofrece dos estados siguientes y dos juegos de salidas, entre los cuales elige el valor de esa entrada.

Para ello se diseñó una carta ASM de nueve estados, cuatro entradas y cuatro salidas (dos con lógica estándar y dos con lógica negada), con salidas condicionales en varias decisiones; se determinó el tamaño de memoria requerido, se obtuvo su contenido completo y se implementó el sistema en VHDL sobre la tarjeta DE10-Lite mediante Quartus Prime, verificando su funcionamiento por simulación y en hardware.

Introducción

Una carta ASM (*Algorithmic State Machine*) representa gráficamente el comportamiento de un circuito secuencial: los estados se dibujan como cajas rectangulares, las condiciones de transición como cajas de decisión y las salidas que dependen a la vez del estado y de las entradas como cajas ovaladas o salidas condicionales. Estas últimas corresponden al modelo de Mealy, en el que las salidas pueden cambiar sin que medie un flanco de *clock*, a diferencia del modelo de Moore, donde son función únicamente del estado presente [2].

El método empleado en esta práctica, el **direccionamiento entrada-estado**, se restringe a cartas ASM que sensan una sola entrada por estado [1]. Esa restricción es justamente lo que permite reducir el tamaño de la memoria: mientras que en el direccionamiento por trayectoria la dirección se formaba concatenando el estado presente con *todas* las variables de entrada, aquí la dirección se compone exclusivamente del estado presente. La información sobre las entradas se traslada al contenido de la palabra mediante un campo nuevo, la **parte de prueba**, que guarda la representación binaria de la variable que debe sensarse en ese estado. Un selector de entradas, implementado como un multiplexor, usa ese campo para elegir la señal correspondiente.

En consecuencia, la palabra de memoria contiene dos estados siguientes en lugar de uno: la **liga falsa**, que se toma cuando la entrada sensada vale cero, y la **liga verdadera**, que se toma cuando vale uno. Un segundo multiplexor, el selector de liga, escoge entre ambas según el valor entregado por el selector de entradas, y el resultado se almacena en el registro de estado en el siguiente flanco

de reloj.

Para manejar salidas condicionales, el diagrama de bloques básico debe modificarse desdoblando también el campo de salidas en dos: el campo de **salidas falsas**, activo cuando la variable sensada vale cero, y el de **salidas verdaderas**, activo cuando vale uno [1]. Un tercer multiplexor selecciona entre ellos con el mismo criterio. Con esta estructura, el tamaño de la memoria queda determinado por

$$\text{Memoria} = 2^n \times m \tag{1}$$

donde n es el número de bits de dirección, que ahora equivale únicamente a los bits de estado, y m el número de bits por localidad, igual a la suma de los bits de prueba, de liga falsa, de liga verdadera, de salidas falsas y de salidas verdaderas.

La ventaja frente al direccionamiento por trayectoria es que el número de localidades deja de crecer exponencialmente con la cantidad de entradas: agregar una entrada más solo ensancha el campo de prueba en lugar de duplicar la memoria. El precio es la restricción de una sola prueba por estado, que obliga a desdoblar en estados auxiliares cualquier estado que necesite evaluar dos condiciones.

Desarrollo

Planteamiento: carta ASM

Se diseñó la carta ASM de la figura 1, que cumple con las características solicitadas: nueve estados, cuatro entradas (Q_1 a Q_4) y cuatro salidas, de las cuales S_1 y S_2 operan con lógica estándar (activas en 1) y S_3 y S_4 con lógica negada (activas en 0, lo que se indica con una barra sobre el nombre). La carta incluye cuatro salidas condicionales, por encima de las tres que pide el manual, y respeta la restricción del método: ningún estado sensa más de una entrada. El diseño preliminar a mano se muestra en la figura 2.

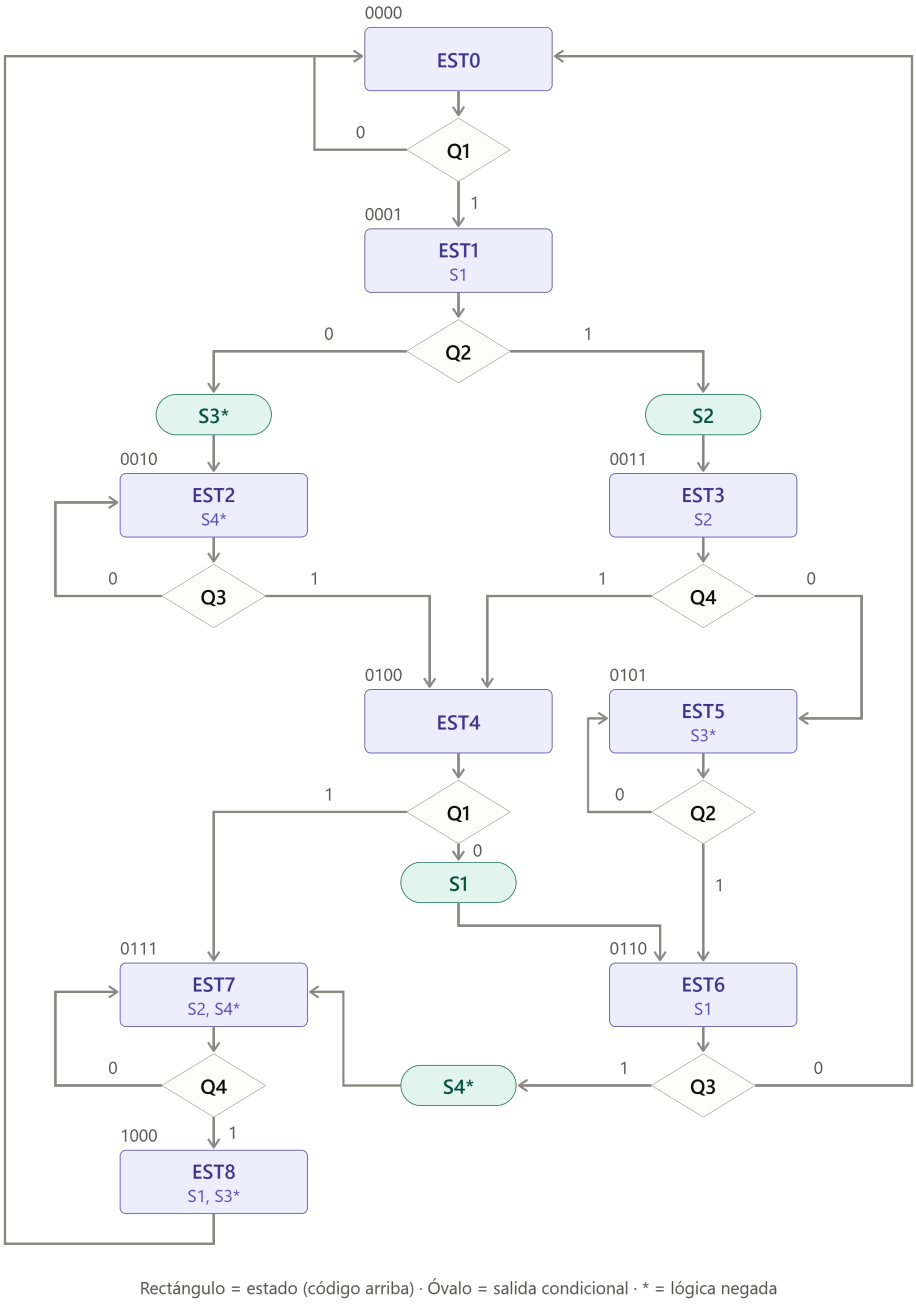


Figura 1: Carta ASM propuesta: nueve estados, cuatro entradas y cuatro salidas, con salidas condicionales en las decisiones de EST1, EST4 y EST6.

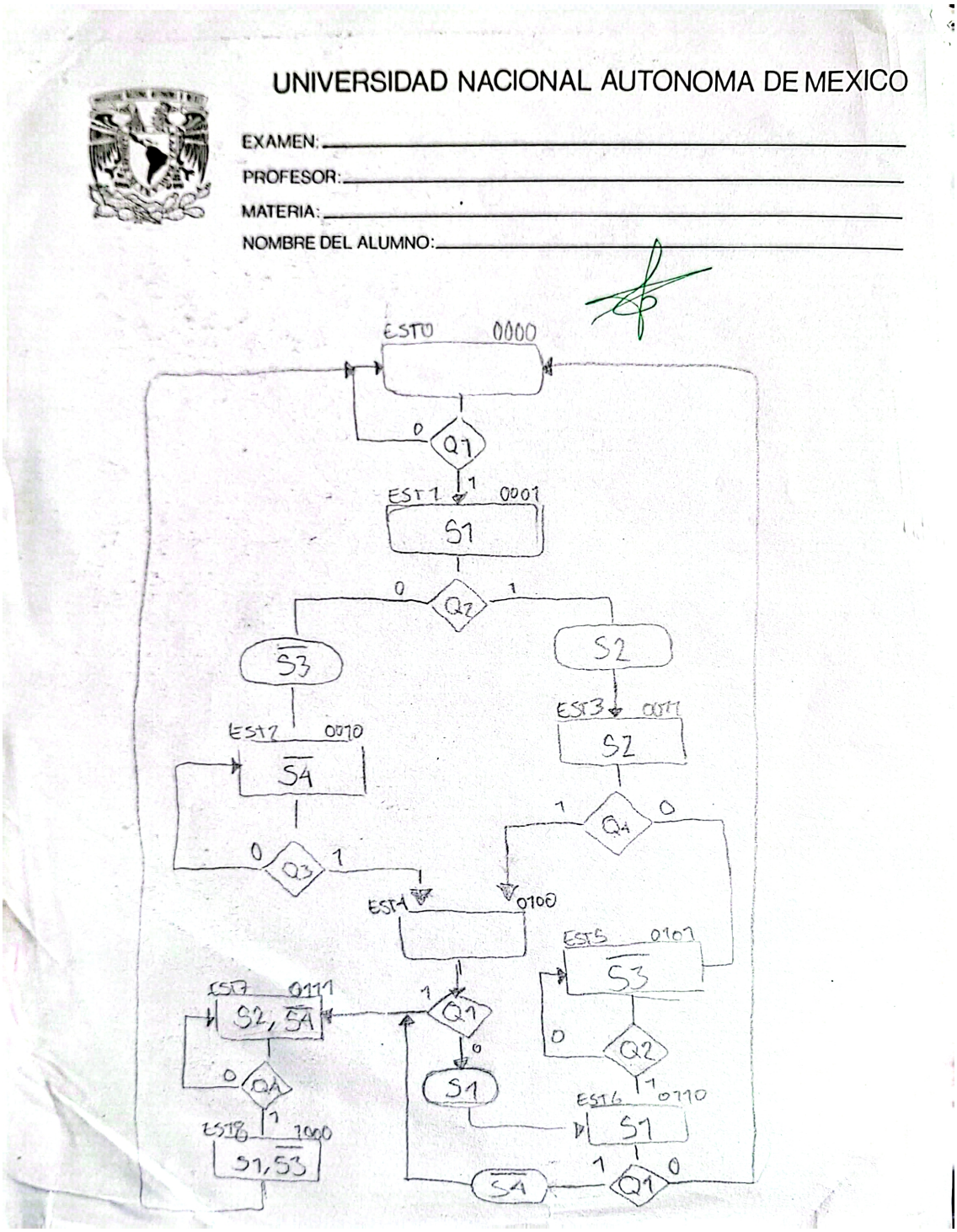


Figura 2: Evidencia del diseño manual de la carta ASM previo a su captura digital, con la asignación de códigos de estado.

Antes de construir la tabla fue necesario asignar una representación binaria tanto a los estados como a las entradas. Con nueve estados se requieren $\lceil \log_2 9 \rceil = 4$ bits, de modo que el espacio de

direcciones abarca 16 localidades y quedan siete códigos sin utilizar. Para las cuatro entradas bastan $\lceil \log_2 4 \rceil = 2$ bits en el campo de prueba. Ambas asignaciones se resumen en las tablas 1 y 2.

Tabla 1: Asignación binaria de los estados, con su salida incondicional y la variable que sensan.

Estado	Código	Salida	Prueba
EST0	`0000`	—	Q_1
EST1	`0001`	S_1	Q_2
EST2	`0010`	$\overline{S_4}$	Q_3
EST3	`0011`	S_2	Q_4
EST4	`0100`	—	Q_1
EST5	`0101`	$\overline{S_3}$	Q_2
EST6	`0110`	S_1	Q_3
EST7	`0111`	S_2, $\overline{S_4}$	Q_4
EST8	`1000`	S_1, $\overline{S_3}$	—

Tabla 2: Asignación binaria de las entradas en el campo de prueba.

Prueba	Entrada
`00`	Q_1
`01`	Q_2
`10`	Q_3
`11`	Q_4

El comportamiento de la máquina es el siguiente. EST0 no activa salidas y permanece en sí mismo mientras $Q_1 = 0$; con $Q_1 = 1$ avanza a EST1. EST1 activa S_1 y sensa Q_2: por la rama falsa activa además la salida condicional $\overline{S_3}$ y transita a EST2, y por la verdadera activa S_2 y transita a EST3. EST2 activa $\overline{S_4}$ y se autolaza mientras $Q_3 = 0$, avanzando a EST4 cuando $Q_3 = 1$. EST3 activa S_2 y, según Q_4, va a EST5 (falsa) o a EST4 (verdadera). EST4 no activa salidas incondicionales y sensa Q_1: con $Q_1 = 0$ activa la condicional S_1 y pasa a EST6, y con $Q_1 = 1$ pasa a EST7. EST5 activa $\overline{S_3}$ y se autolaza mientras $Q_2 = 0$, avanzando a EST6 cuando $Q_2 = 1$. EST6 activa S_1 y sensa Q_3: con $Q_3 = 0$ regresa a EST0, y con $Q_3 = 1$ activa además la condicional $\overline{S_4}$ y pasa a EST7. EST7 activa S_2 y $\overline{S_4}$, se autolaza mientras $Q_4 = 0$ y avanza a EST8 cuando $Q_4 = 1$. Finalmente, EST8 activa S_1 y $\overline{S_3}$ y, al no contener caja de decisión, regresa incondicionalmente a EST0.

Respecto a la codificación de las salidas negadas se conservó el criterio de la práctica anterior: almacenar en memoria el nivel físico de la señal, de modo que el bit de $\overline{S_3}$ o $\overline{S_4}$ vale `0` cuando la salida está activa y `1` cuando está en reposo. Así la palabra leída se conecta directamente a los pines de salida sin inversores intermedios.

Tamaño de la memoria necesaria

Los cálculos manuales de la codificación y del tamaño se muestran en la figura 3. De acuerdo con la ecuación 1, la dirección se forma únicamente con los 4 bits del estado presente, por lo que

$n = 4$ y la memoria tiene $2^4 = 16$ localidades. Cada localidad almacena los cinco campos del método:

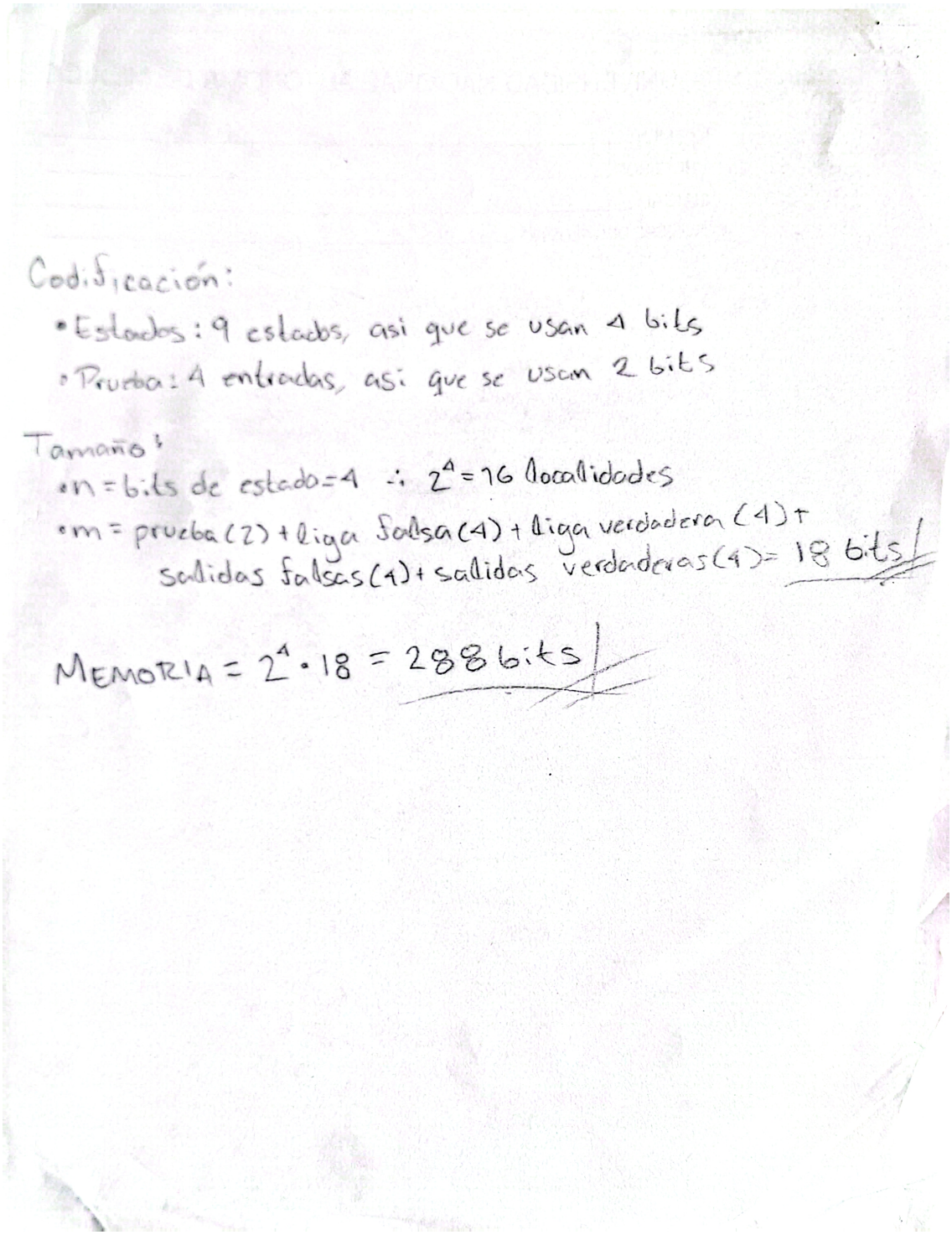


Figura 3: Evidencia del cálculo manual de la codificación y del tamaño de la memoria: 4 bits de estado, 2 bits de prueba, 18 bits por localidad y 288 bits en total.

Campo	Bits
Parte de prueba (4 entradas)	2
Liga falsa (9 estados)	4
Liga verdadera (9 estados)	4
Salidas falsas	4
Salidas verdaderas	4
Total m	**18**

Por lo tanto

$$\text{Memoria} = 2^4 \times 18 = 16 \times 18 = 288 \text{ bits},$$

es decir, 16 localidades de 18 bits cada una. De ellas, nueve corresponden a estados que existen en la carta ASM y las siete restantes (`1001` a `1111`) a códigos no utilizados. Siguiendo la indicación del manual de considerar el llenado de todos los estados representables con los bits asignados, a esas localidades se les dio un contenido explícito de seguridad: ambas ligas apuntan a EST0 y los dos campos de salidas quedan en reposo, de modo que si la máquina alcanzara uno de esos códigos por una condición anómala retornaría al estado inicial en el siguiente flanco de reloj.

Conviene contrastar este resultado con el de la práctica anterior. Con direccionamiento por trayectoria, una carta de nueve estados y cuatro entradas habría necesitado $2^{4+4} = 256$ localidades; el direccionamiento entrada-estado las reduce a 16 a costa de ensanchar cada palabra de 7 a 18 bits, lo que deja la memoria total en 288 bits frente a los 1 792 que habría exigido el método anterior.

Contenido de la memoria

El contenido se obtuvo recorriendo la carta ASM estado por estado. A diferencia de la práctica anterior, no hay que enumerar las combinaciones de entrada: a cada estado le corresponde una sola localidad, cuya dirección es su propio código. La palabra se organiza como

$$\text{dato} = \underbrace{\text{Prueba}}_{(17:16)} \,\|\, \underbrace{\text{Liga falsa}}_{(15:12)} \,\|\, \underbrace{\text{Liga verdadera}}_{(11:8)} \,\|\, \underbrace{\text{Salidas falsas}}_{(7:4)} \,\|\, \underbrace{\text{Salidas verdaderas}}_{(3:0)}$$

En los estados cuya salida es incondicional, los campos de salidas falsas y verdaderas son idénticos; sólo difieren en EST1, EST4 y EST6, que son los que tienen salidas condicionales. En

EST8 y en los estados no utilizados la parte de prueba es indiferente, puesto que ambas ligas coinciden; se escribió `00` para dejar el campo en un valor definido en lugar de un *don't care*. La tabla 3 presenta las 16 localidades.

Tabla 3: Contenido de memoria obtenido con direccionamiento entrada-estado. Las direcciones 9 a 15 corresponden a códigos de estado no utilizados: ambas ligas apuntan a EST0 y las salidas quedan en reposo.

		Prueba				**Salidas falsas**				**Salidas verdaderas**			
Dir.	**Edo.**	bin	var	**LF**	**LV**	S_1	S_2	$\overline{S_3}$	$\overline{S_4}$	S_1	S_2	$\overline{S_3}$	$\overline{S_4}$
0	EST0	00	Q_1	0000	0001	0	0	1	1	0	0	1	1
1	EST1	01	Q_2	0010	0011	1	0	0	1	1	1	1	1
2	EST2	10	Q_3	0010	0100	0	0	1	0	0	0	1	0
3	EST3	11	Q_4	0101	0100	0	1	1	1	0	1	1	1
4	EST4	00	Q_1	0110	0111	1	0	1	1	0	0	1	1
5	EST5	01	Q_2	0101	0110	0	0	0	1	0	0	0	1
6	EST6	10	Q_3	0000	0111	1	0	1	1	1	0	1	0
7	EST7	11	Q_4	0111	1000	0	1	1	0	0	1	1	0
8	EST8	00	—	0000	0000	1	0	0	1	1	0	0	1
9	—	00	—	0000	0000	0	0	1	1	0	0	1	1
10	—	00	—	0000	0000	0	0	1	1	0	0	1	1
11	—	00	—	0000	0000	0	0	1	1	0	0	1	1
12	—	00	—	0000	0000	0	0	1	1	0	0	1	1
13	—	00	—	0000	0000	0	0	1	1	0	0	1	1
14	—	00	—	0000	0000	0	0	1	1	0	0	1	1
15	—	00	—	0000	0000	0	0	1	1	0	0	1	1

Implementación en VHDL

El sistema se describió en VHDL siguiendo el diagrama de bloques de la figura 3 del manual, repartido en siete módulos más una entidad de nivel superior. El lazo de datos es el siguiente: el registro de estado entrega el estado presente, que direcciona la memoria; la palabra leída se separa en sus cinco campos; el selector de entradas toma la parte de prueba y engancha una de las cuatro entradas; y el valor de esa entrada gobierna simultáneamente al selector de liga, cuya salida regresa al registro, y al selector de salidas, que maneja los pines S_1 a S_4.

El módulo `memory` implementa la ROM de 16 palabras de 18 bits. Cada asignación corresponde a un renglón de la tabla 3 y está comentada con el estado y la variable que sensa.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
5
6  entity memory is
7    Port( dir : in STD_LOGIC_VECTOR(3 downto 0); --Tamaño de direccion (solo el
         edo. presente)
8         data : out STD_LOGIC_VECTOR(17 downto 0) --Tamaño de memoria por
             registro
9         );
10 end memory;
11
12 architecture Behavioral of memory is
13   type mem is array(0 to 15) of std_logic_vector(17 downto 0); --tamaño de
         memoria por registro
14   signal internal_mem : mem;
15
16 begin
17   --Llenado de memoria
18   --  Palabra = Prueba(2) & LigaFalsa(4) & LigaVerdadera(4) & SalFalsas(4) &
         SalVerdaderas(4)
19   --  Memoria = 2^4 x 18 = 288 bits
20   --
21   --  Prueba:  00=Q1  01=Q2  10=Q3  11=Q4
22   --  Salidas: S1 S2 S3 S4   (S1,S2 logica estandar: 1=activa)
23   --                         (S3,S4 logica negada:   0=activa)
24   --
25   --  En EST8 y en los estados no usados la prueba es indiferente (LF=LV);
26   --  se escribe 00 para dejar el campo en un valor definido.
27
28   --                       Prueba  LigaF    LigaV    SalFalsas  SalVerdaderas
29   internal_mem( 0) <= "00" & "0000" & "0001" & "0011" & "0011"; --EST0: sin
         salidas, prueba Q1
30   internal_mem( 1) <= "01" & "0010" & "0011" & "1001" & "1111"; --EST1: S1,
         prueba Q2, cond. S3*/S2
31   internal_mem( 2) <= "10" & "0010" & "0100" & "0010" & "0010"; --EST2: S4*,
         prueba Q3
32   internal_mem( 3) <= "11" & "0101" & "0100" & "0111" & "0111"; --EST3: S2,
         prueba Q4
33   internal_mem( 4) <= "00" & "0110" & "0111" & "1011" & "0011"; --EST4: sin
         salidas, prueba Q1, cond. S1
34   internal_mem( 5) <= "01" & "0101" & "0110" & "0001" & "0001"; --EST5: S3*,
         prueba Q2
35   internal_mem( 6) <= "10" & "0000" & "0111" & "1011" & "1010"; --EST6: S1,
         prueba Q3, cond. S4*
36   internal_mem( 7) <= "11" & "0111" & "1000" & "0110" & "0110"; --EST7: S2 y
         S4*, prueba Q4
```

```
  internal_mem( 8) <= "00" & "0000" & "0000" & "1001" & "1001"; --EST8: S1 y S3*, sin prueba -> EST0

  --Estados no utilizados (1001 a 1111): regresan a EST0 con las salidas en reposo
  internal_mem( 9) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(10) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(11) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(12) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(13) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(14) <= "00" & "0000" & "0000" & "0011" & "0011";
  internal_mem(15) <= "00" & "0000" & "0000" & "0011" & "0011";

  process(dir, internal_mem)
  begin
    data <= internal_mem(conv_integer(unsigned(dir)));
  end process;
end Behavioral;
```

Figura 4: Código VHDL de la memoria con las 16 localidades del direccionamiento entrada-estado.

El módulo `divisor_datos` separa la palabra leída en los cinco campos del método.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divisor_datos is
  Port( entrada : in STD_LOGIC_VECTOR(17 downto 0); --Bits de memoria por registro
       -- Complementa con cada campo de memoria
       prueba : out STD_LOGIC_VECTOR(1 downto 0);      --Entrada a sensar
       liga_f : out STD_LOGIC_VECTOR(3 downto 0);      --Liga falsa
       liga_v : out STD_LOGIC_VECTOR(3 downto 0);      --Liga verdadera
       salidas_f : out STD_LOGIC_VECTOR(3 downto 0);   --Salidas falsas
       salidas_v : out STD_LOGIC_VECTOR(3 downto 0)    --Salidas verdaderas
       );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
  process (entrada)
  begin
    --Asigna cada parte de la memoria a los bits correspondientes de entrada
```

```
    prueba    <= entrada(17 downto 16);
    liga_f    <= entrada(15 downto 12);
    liga_v    <= entrada(11 downto 8);
    salidas_f <= entrada(7 downto 4);
    salidas_v <= entrada(3 downto 0);
  end process;
end Behavioral;
```

Figura 5: Código VHDL del divisor que separa la palabra en prueba, ligas y salidas.

El selector de entradas, `mux4`, es el módulo que distingue a este método: toma los dos bits de la parte de prueba y conecta a su salida únicamente la entrada indicada, dejando las otras tres sin efecto sobre el comportamiento de la máquina en ese estado.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--Multiplexor de entradas
entity mux4 is
  Port( sel : in STD_LOGIC_VECTOR(1 downto 0);
      --Defina sus entradas
      Q1 : in STD_LOGIC;
      Q2 : in STD_LOGIC;
      Q3 : in STD_LOGIC;
      Q4 : in STD_LOGIC;
      O : out STD_LOGIC
      );
end mux4;

architecture Behavioral of mux4 is
begin
  process (sel, Q1, Q2, Q3, Q4)
  begin
    -- Asigne a "O" la entrada dependiendo del selector
    case sel is
      when "00" => O <= Q1;
      when "01" => O <= Q2;
      when "10" => O <= Q3;
      when others => O <= Q4;
    end case;
  end process;
end Behavioral;
```

Figura 6: Código VHDL del selector de entradas gobernado por la parte de prueba.

Los dos multiplexores de 2 a 1 eligen, con el valor de la entrada sensada, entre liga falsa y verdadera y entre salidas falsas y verdaderas.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--Multiplexor de liga
entity mux2_3 is
  Port( sel : in STD_LOGIC;
       --Defina las ligas
       liga_f : in STD_LOGIC_VECTOR(3 downto 0);
       liga_v : in STD_LOGIC_VECTOR(3 downto 0);
       O : out STD_LOGIC_VECTOR(3 downto 0)
       );
end mux2_3;

architecture Behavioral of mux2_3 is
begin
  process (sel, liga_f, liga_v)
  begin
    --Asigne a "O" la liga dependiendo del selector
    if sel = '0' then
      O <= liga_f;
    else
      O <= liga_v;
    end if;
  end process;
end Behavioral;
```

Figura 7: Código VHDL del selector de liga.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--Multiplexor de Salidas
entity mux2_4 is
  Port( sel : in STD_LOGIC;
       --Defina las salidas verdaderas y falsas
       salidas_f : in STD_LOGIC_VECTOR(3 downto 0);
       salidas_v : in STD_LOGIC_VECTOR(3 downto 0);
       O : out STD_LOGIC_VECTOR(3 downto 0)
       );
end mux2_4;

architecture Behavioral of mux2_4 is
```

```
begin
  process (sel, salidas_f, salidas_v)
  begin
    --Asigne a "0" las salidas dependiendo del selector
    if sel = '0' then
      O <= salidas_f;
    else
      O <= salidas_v;
    end if;
  end process;
end Behavioral;
```

Figura 8: Código VHDL del selector de salidas, que permite las salidas condicionales.

El registro de estado es el único elemento secuencial del lazo. Tiene *reset* asíncrono activo en bajo que fuerza EST0 y, en cada flanco de subida, captura la liga seleccionada. Se amplió a cuatro bits respecto al esqueleto original, como lo permite el propio comentario del archivo, para alojar los nueve estados.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register3 is
  Port( CLK : in STD_LOGIC;
      RESET : in STD_LOGIC;
      DATA_IN : in STD_LOGIC_VECTOR(3 downto 0); --Liga
      DATA_OUT : out STD_LOGIC_VECTOR(3 downto 0)--Edo Presente
      );
end register3;

architecture Behavioral of register3 is
  signal internal_value : std_logic_vector(3 downto 0) := B"0000"; --9
     estados: se requieren 4 bits
begin
  process (CLK, RESET, DATA_IN)
  begin
    if RESET = '0' then
      internal_value <= B"0000";
    elsif rising_edge (CLK) then
      internal_value <= DATA_IN;
    end if;
  end process;

  process(internal_value)
```

```
  begin
    DATA_OUT <= internal_value;
  end process;
end Behavioral;
```

Figura 9: Código VHDL del registro que almacena el estado presente.

Dado que el oscilador de la tarjeta opera a 50 MHz, las transiciones resultarían imperceptibles. El divisor de frecuencia reinicia su contador al alcanzar `X"2FAF080"` (50 000 000) y toma la salida del bit 25, lo que produce un periodo de aproximadamente 1.0 s: una transición de estado por segundo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity div_frec is
  Port (reloj : in std_logic;
        div_clk : out std_logic);
end div_frec;

architecture Behavioral of div_frec is
begin
  process (reloj)
  variable cuenta : std_logic_vector (27 downto 0) := X"0000000";
  begin
    if rising_edge (reloj) then

      --if cuenta 0 "4000000" then
      if cuenta = X"2FAF080" then
        cuenta := X"0000000";
      else
        cuenta := cuenta+1;
      end if;
    end if;
    div_clk <= cuenta(25);
  end process;
end Behavioral;
```

Figura 10: Código VHDL del divisor de frecuencia que fija la velocidad de la máquina.

Finalmente, la entidad `practica4` instancia los siete módulos y los interconecta, además de exponer el estado presente hacia los LEDs para poder observarlo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
3
4  -- Practica 4: Construccion de Maquinas de Estados usando Memorias
5  -- Direccionamiento Entrada-Estado
6  --
7  -- La direccion de memoria es UNICAMENTE el estado presente (4 bits, 9
       estados). La palabra
8  -- leida trae la prueba, las dos ligas y los dos juegos de salidas; el
       selector de
9  -- entradas (mux4) elige la variable a sensar y su valor gobierna los dos mux
       de
10 -- 2 a 1 que escogen liga y salidas.
11 --
12 --                                  +-----------+
13 --   Q1..Q4 -------------------->   |   mux4    |--- sel ---+-------------+
14 --                          +------->|  entradas |           |             |
15 --                          | prueba +-----------+           v             v
16 --   +-----------+          |                           +----------+
       +----------+
17 --   | register3 |--dir-->| memory |--16--> divisor --| mux2_3  |   | mux2_4
       |
18 --   |  (edo)    |          +--------+        _datos    |  liga   |   | salidas
       |
19 --   +-----^-----+                                     +----+----+
       +----+----+
20 --         |                                               |             |
21 --         +------------------- liga ----------------------+             v
22 --                                                                    S1..S4
23
24 entity practica4 is
25   Port( CLK50 : in STD_LOGIC;                          --Reloj de 50 MHz de la
       tarjeta
26       RESET : in STD_LOGIC;                          --Activo en bajo (KEY0)
27       Q1 : in STD_LOGIC;
28       Q2 : in STD_LOGIC;
29       Q3 : in STD_LOGIC;
30       Q4 : in STD_LOGIC;
31       S1 : out STD_LOGIC;                            --Logica estandar
32       S2 : out STD_LOGIC;                            --Logica estandar
33       S3 : out STD_LOGIC;                            --Logica negada (activa en
           0)
34       S4 : out STD_LOGIC;                            --Logica negada (activa en
           0)
35       EDO : out STD_LOGIC_VECTOR(3 downto 0)         --Estado presente
36       );
```

```
37 end practica4;
38
39 architecture Estructural of practica4 is
40
41   component div_frec is
42     Port( reloj : in std_logic;
43         div_clk : out std_logic);
44   end component;
45
46   component register3 is
47     Port( CLK : in STD_LOGIC;
48         RESET : in STD_LOGIC;
49         DATA_IN : in STD_LOGIC_VECTOR(3 downto 0);
50         DATA_OUT : out STD_LOGIC_VECTOR(3 downto 0));
51   end component;
52
53   component memory is
54     Port( dir : in STD_LOGIC_VECTOR(3 downto 0);
55         data : out STD_LOGIC_VECTOR(17 downto 0));
56   end component;
57
58   component divisor_datos is
59     Port( entrada : in STD_LOGIC_VECTOR(17 downto 0);
60         prueba : out STD_LOGIC_VECTOR(1 downto 0);
61         liga_f : out STD_LOGIC_VECTOR(3 downto 0);
62         liga_v : out STD_LOGIC_VECTOR(3 downto 0);
63         salidas_f : out STD_LOGIC_VECTOR(3 downto 0);
64         salidas_v : out STD_LOGIC_VECTOR(3 downto 0));
65   end component;
66
67   component mux4 is
68     Port( sel : in STD_LOGIC_VECTOR(1 downto 0);
69         Q1 : in STD_LOGIC;
70         Q2 : in STD_LOGIC;
71         Q3 : in STD_LOGIC;
72         Q4 : in STD_LOGIC;
73         O : out STD_LOGIC);
74   end component;
75
76   component mux2_3 is
77     Port( sel : in STD_LOGIC;
78         liga_f : in STD_LOGIC_VECTOR(3 downto 0);
79         liga_v : in STD_LOGIC_VECTOR(3 downto 0);
80         O : out STD_LOGIC_VECTOR(3 downto 0));
```

```
81    end component;
82
83    component mux2_4 is
84      Port( sel : in STD_LOGIC;
85          salidas_f : in STD_LOGIC_VECTOR(3 downto 0);
86          salidas_v : in STD_LOGIC_VECTOR(3 downto 0);
87          O : out STD_LOGIC_VECTOR(3 downto 0));
88    end component;
89
90    signal clk_fsm : std_logic;
91    signal edo_pres : std_logic_vector(3 downto 0);
92    signal palabra_mem : std_logic_vector(17 downto 0);  --Contenido de memoria
        leido
93    signal prueba : std_logic_vector(1 downto 0);
94    signal liga_f : std_logic_vector(3 downto 0);
95    signal liga_v : std_logic_vector(3 downto 0);
96    signal salidas_f : std_logic_vector(3 downto 0);
97    signal salidas_v : std_logic_vector(3 downto 0);
98    signal sel_entrada : std_logic;                      --Valor de la variable
        sensada
99    signal liga : std_logic_vector(3 downto 0);
100   signal salidas : std_logic_vector(3 downto 0);
101
102 begin
103
104   U1 : div_frec
105     port map( reloj => CLK50,
106           div_clk => clk_fsm);
107
108   U2 : register3
109     port map( CLK => clk_fsm,
110           RESET => RESET,
111           DATA_IN => liga,
112           DATA_OUT => edo_pres);
113
114   U3 : memory
115     port map( dir => edo_pres,
116           data => palabra_mem);
117
118   U4 : divisor_datos
119     port map( entrada => palabra_mem,
120           prueba => prueba,
121           liga_f => liga_f,
122           liga_v => liga_v,
```

```
123             salidas_f => salidas_f,
124             salidas_v => salidas_v);
125
126   U5 : mux4
127     port map( sel => prueba,
128             Q1 => Q1,
129             Q2 => Q2,
130             Q3 => Q3,
131             Q4 => Q4,
132             O => sel_entrada);
133
134   U6 : mux2_3
135     port map( sel => sel_entrada,
136             liga_f => liga_f,
137             liga_v => liga_v,
138             O => liga);
139
140   U7 : mux2_4
141     port map( sel => sel_entrada,
142             salidas_f => salidas_f,
143             salidas_v => salidas_v,
144             O => salidas);
145
146   S1 <= salidas(3);
147   S2 <= salidas(2);
148   S3 <= salidas(1);
149   S4 <= salidas(0);
150   EDO <= edo_pres;
151
152 end Estructural;
```

Figura 11: Código VHDL de la entidad de nivel superior que interconecta los módulos.

Como apoyo visual, la figura 12 presenta la misma interconexión vista como esquemático de bloques en Quartus.

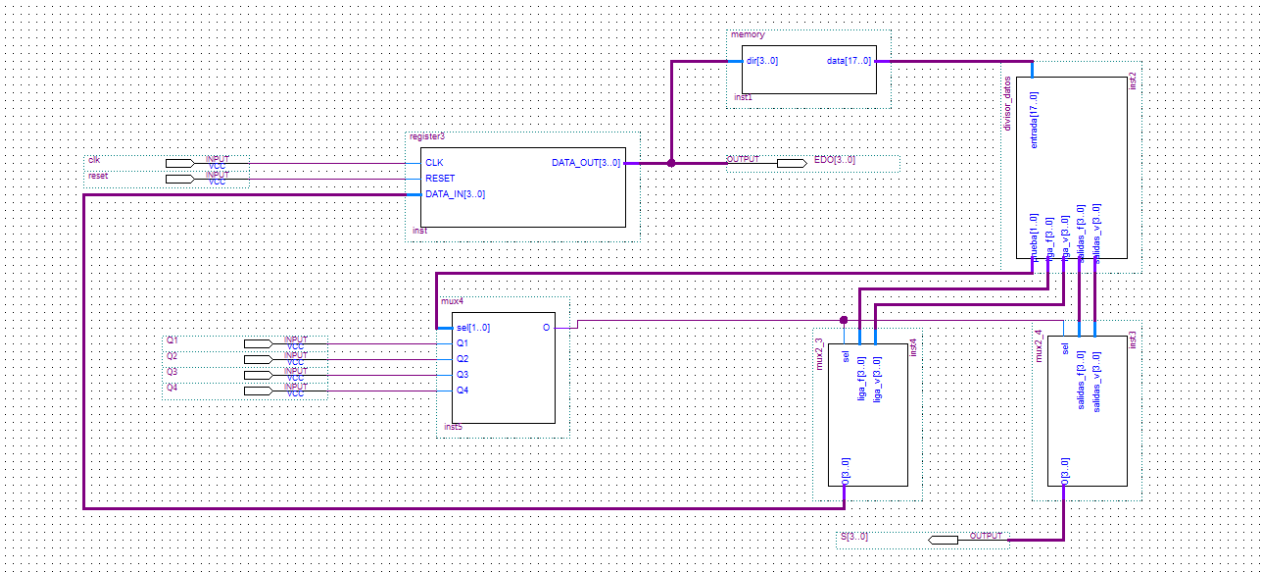


Figura 12: Esquemático en Quartus con la interconexión de los módulos.

Asignación de pines

Para la implementación en la tarjeta DE10-Lite (dispositivo MAX 10 10M50DAF484C7G) se realizó la asignación de la tabla 4. Las cuatro entradas se asignaron a interruptores deslizables, el *reset* al botón `KEY0` (que la tarjeta entrega en bajo al ser presionado, lo cual coincide con la polaridad del registro), las cuatro salidas a `LEDR0`–`LEDR3` y los cuatro bits del estado presente a `LEDR6`–`LEDR9`, ordenados de modo que el más significativo quede a la izquierda.

Tabla 4: Asignación de pines en la tarjeta DE10-Lite.

Señal	Pin	Recurso	Función
`CLK50`	`PIN_P11`	`MAX10_CLK1_50`	Reloj de 50 MHz
`RESET`	`PIN_B8`	`KEY0`	*Reset* activo en bajo
`Q1`	`PIN_C10`	`SW0`	Entrada Q_1
`Q2`	`PIN_C11`	`SW1`	Entrada Q_2
`Q3`	`PIN_D12`	`SW2`	Entrada Q_3
`Q4`	`PIN_C12`	`SW3`	Entrada Q_4
`S1`	`PIN_A8`	`LEDR0`	Salida S_1 (activa en 1)
`S2`	`PIN_A9`	`LEDR1`	Salida S_2 (activa en 1)
`S3`	`PIN_A10`	`LEDR2`	Salida $\overline{S_3}$ (activa en 0)
`S4`	`PIN_B10`	`LEDR3`	Salida $\overline{S_4}$ (activa en 0)
`EDO[3]`	`PIN_B11`	`LEDR9`	Bit 3 del estado presente
`EDO[2]`	`PIN_A11`	`LEDR8`	Bit 2 del estado presente
`EDO[1]`	`PIN_D14`	`LEDR7`	Bit 1 del estado presente
`EDO[0]`	`PIN_E14`	`LEDR6`	Bit 0 del estado presente

Análisis de resultados

El proyecto se compiló en Quartus Prime 18.1.0 Lite Edition sin errores. La tabla 5 resume los recursos utilizados y el desempeño reportado.

Tabla 5: Resumen de compilación del proyecto en Quartus Prime 18.1.

Parámetro	**Valor**
Elementos lógicos	71 / 49 760 (< 1 %)
Registros dedicados	32 / 49 760 (< 1 %)
Pines de E/S	14 / 360 (4 %)
Bits de memoria embebida	0 / 1 677 312 (0 %)
F_{max} (modelo 1200 mV, 85 °C)	189.68 MHz
Errores de compilación	0

Dos cifras de la tabla 5 merecen comentario. La primera es el conteo de 32 registros, que coincide con lo esperado: 28 corresponden al contador del divisor de frecuencia y los 4 restantes al registro de estado, lo que confirma que el único elemento secuencial del lazo de la máquina es ese registro. La segunda es que la herramienta reporta cero bits de memoria embebida: la ROM de 288 bits no se mapeó a los bloques M9K del dispositivo sino que se sintetizó como lógica combinacional dentro de los 71 elementos lógicos, lo cual es razonable dado su tamaño y no altera el comportamiento del circuito. Los 189.68 MHz de frecuencia máxima superan ampliamente los 50 MHz del oscilador, de modo que el diseño opera con holgura; la advertencia de incumplimiento temporal que emite el analizador se debe únicamente a que el proyecto no incluye un archivo `.sdc`, y en ausencia de restricciones Quartus asume un requisito por omisión de 1 GHz.

Simulación funcional

Antes de programar la tarjeta se verificó el diseño mediante simulación funcional con un *testbench* autoverificable. El banco de pruebas aplica una secuencia de 22 vectores de entrada que recorre los nueve estados y ambas ramas de cada decisión, incluidas las cuatro salidas condicionales y el paso EST7 → EST8 → EST0. En cada transición compara el estado resultante con el previsto por la carta ASM mediante sentencias `assert`, y al finalizar reporta que las 22 transiciones coinciden con la carta. La simulación concluyó sin discrepancias.

La simulación se compiló y ejecutó con GHDL, y la forma de onda se visualizó con GTKWave a partir del archivo `sim.vcd`. El *testbench* instancia la cadena funcional completa (memoria, divisor de datos, selector de entradas, los dos multiplexores de 2 a 1 y el registro de estado) manejando el registro con un reloj directo de 10 ns. Se omite deliberadamente el divisor de frecuencia, ya que ese módulo sólo fija la velocidad perceptible en la tarjeta y simularlo exigiría 2^{25} ciclos por cada flanco.

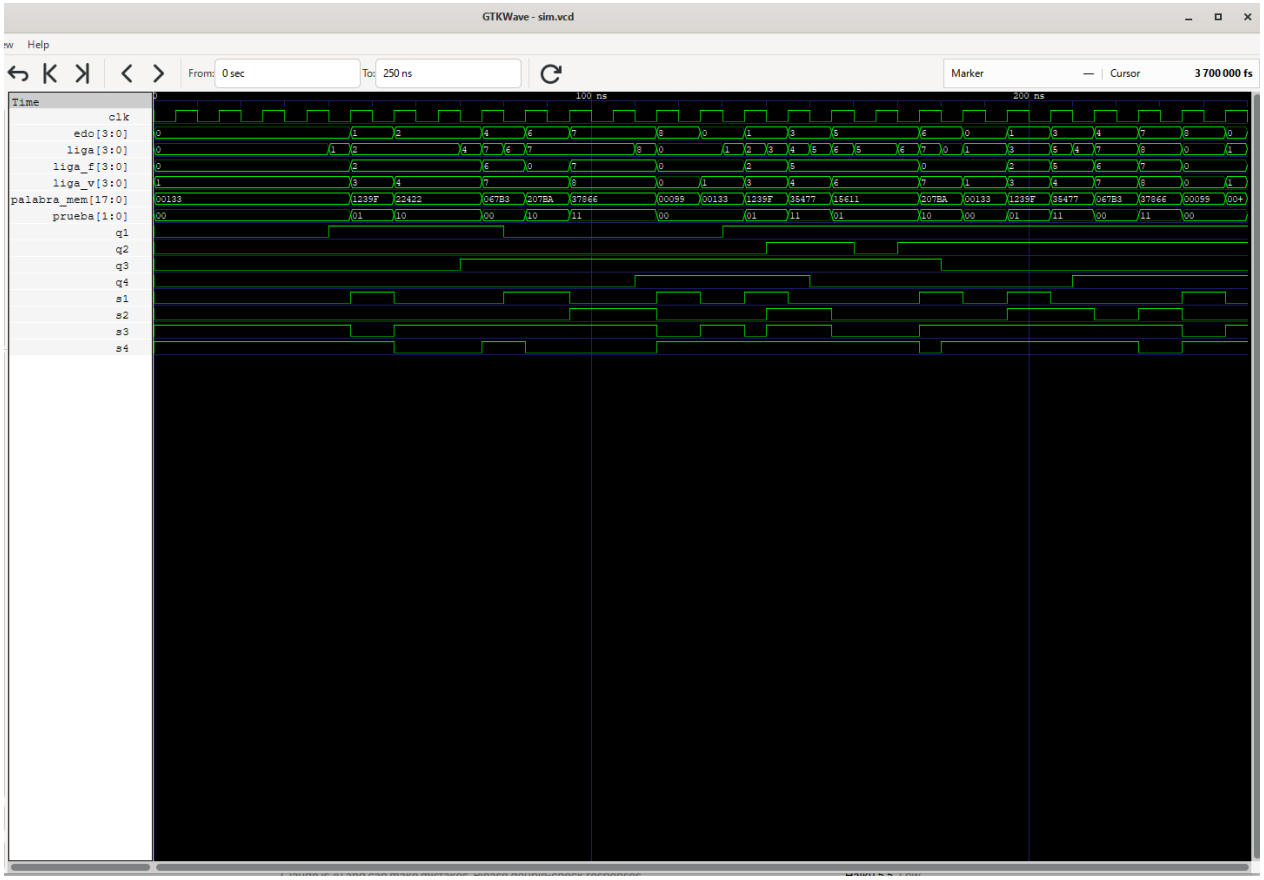


Figura 13: Forma de onda de la simulación funcional. Se muestran el reloj, el estado presente (`edo`), la liga seleccionada y sus versiones falsa y verdadera, la palabra leída de memoria (`palabra_mem`), la parte de prueba, las entradas Q_1 a Q_4 y las salidas S_1 a S_4, durante las 22 transiciones (0 a 250 ns).

En la figura se observa que el estado presente recorre los nueve estados de la carta y regresa a EST0 al final del recorrido. Cada cambio de estado coincide con un flanco de reloj, y las salidas S_1 a S_4 se activan según la rama tomada en cada decisión.

Verificación en la tarjeta

Durante la sesión de laboratorio se programó la DE10-Lite con el diseño y se observó su comportamiento. Con los cuatro interruptores en bajo, la máquina permanece en EST0. Esto es consistente con la liga falsa de la dirección 0, que apunta a sí misma mientras $Q_1 = 0$.

Al activar Q_1, $Q_2 = 0$, $Q_3 = 1$ y $Q_4 = 1$, la máquina recorre el lazo EST0→EST1→EST2→EST4→EST7→EST8→EST0. Cada transición corresponde a una liga de la tabla 3: EST1 toma la liga falsa porque $Q_2 = 0$, EST2 toma la liga verdadera porque $Q_3 = 1$, EST4 toma la liga verdadera porque $Q_1 = 1$, EST7 toma la liga verdadera porque $Q_4 = 1$, y EST8 regresa a EST0 de forma incondicional.

Al detenerse en un estado que se autolaza (EST2, EST5 o EST7), los interruptores que no corres-

ponden a su parte de prueba no producen ningún efecto. Esto muestra que el selector de entradas sensa únicamente la variable indicada por la palabra almacenada, y es la diferencia principal con el direccionamiento por trayectoria.
Por último, `LEDR2` y `LEDR3` permanecen encendidos en reposo, ya que la salida negada almacena un 1 cuando está inactiva. Se apagan al activarse $\overline{S_3}$ o $\overline{S_4}$, en los estados donde la carta ASM las activa.

Conclusiones individuales

Omar Alfonso Zendejas Labias

Se cumplió el objetivo de la práctica. La máquina de nueve estados quedó implementada con una memoria de 16 localidades de 18 bits y funcionó tanto en simulación como en la tarjeta siguiendo la carta ASM. Lo que más me sirvió fue comparar este método con el de la práctica anterior: aquí la dirección es nada más el estado presente, y por eso la memoria baja de las 256 localidades que habría pedido el direccionamiento por trayectoria a solo 16, aunque cada palabra pase de 7 a 18 bits. En la tarjeta eso se nota de inmediato, porque estando en un estado que se autolaza los interruptores que no corresponden a su parte de prueba no hacen absolutamente nada; ahí se entiende para qué sirve el selector de entradas. También me quedó claro por qué el método exige una sola prueba por estado, ya que la palabra solo tiene espacio para dos ligas.

Emiliano García Olvera

Considero que se cumplió el objetivo de la práctica. Con el direccionamiento entrada-estado se construyó una máquina de nueve estados a partir de la carta ASM, usando una memoria de 16 localidades de 18 bits (tabla 3), lo que da un total de 288 bits. Frente al direccionamiento por trayectoria, que habría requerido 256 localidades y 1 792 bits, la reducción es considerable. La tabla 5 muestra que el diseño compila sin errores, utiliza menos del 1 % de los elementos lógicos del dispositivo y alcanza una frecuencia máxima de 189.68 MHz, por encima de los 50 MHz del oscilador. La simulación funcional con la secuencia de 22 vectores no presentó discrepancias, lo que confirma que cada transición coincide con la carta ASM. Lo que más me aportó la práctica fue entender por qué este método exige una sola prueba por estado: la palabra sólo tiene espacio para dos ligas, y ese límite es el costo que se paga por reducir el tamaño de la memoria.

Bibliografía

[1] J. Savage Carmona, G. Vázquez y N. E. Chávez Rodríguez, *Diseño de microprocesadores*. Ciudad de México, México: Facultad de Ingeniería, Universidad Nacional Autónoma de Méxi-

co, 2017. [En línea]. Disponible: http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf

[2] M. M. Mano y M. D. Ciletti, *Diseño digital*, 5a ed. Naucalpan de Juárez, México: Pearson Educación, 2013.